



Mi Restaurante VIP

Sistema SaaS de gestion para restaurantes

Documentacion tecnica y guia de instalacion

Tecnologia del sistema - Funcionalidades - Instalacion local
Hosting recomendado - Despliegue paso a paso

1 Tecnología del sistema

El sistema es una aplicación web full-stack construida sobre el framework **Laravel** con una arquitectura **multi-tenant** (cada restaurante opera aislado) y control de acceso por roles.

Backend

PHP 8.2+ con Laravel 12

Base de datos

MySQL 8 (base: saas_restaurante)

Vistas

Blade + Tailwind CSS 3.4

Interactividad

Alpine.js 3 (sin recargar)

Graficos

ApexCharts

Empaquetado

Vite 5

Autenticacion

Sesiones Laravel + roles

Arquitectura

Multi-tenant por restaurante

Roles

superadmin, admin, cajero, mesero, cocina

Servidor web

Apache / Nginx + PHP-FPM

2 Funcionalidades del sistema

El sistema cubre la operación completa de un restaurante y la administración del SaaS:

Operación del salón

- Punto de Venta (POS) con IGV, descuentos y múltiples pagos
- Gestión de mesas en tiempo real
- Reservas con asignación de mesa
- Pedidos y cambios de estado
- Cocina (KDS): tablero en vivo de comandas
- Caja: apertura, movimientos, cierre y arqueo

Carta, productos e inventario

- Categorías y carta/menu
- Carta digital por código QR (pública)
- Pedidos online del cliente desde el QR
- Recetas: insumos por producto y costo real
- Inventario de insumos con alertas de stock
- Kardex: entradas/salidas y descuento automático

Clientes y marketing

- Base de clientes
- Fidelización por puntos (ganar y canjear)
- Promociones y cupones (porcentaje o monto)
- Aplicación de promociones en el POS

Reportes y comprobantes

- Ventas por día y por método de pago
- Productos más vendidos
- Rentabilidad por plato (margen y ganancia)
- Exportación a Excel (CSV) y PDF
- Ticket de venta imprimible (80 mm)

Administracion del restaurante

- Usuarios y roles
- Suscripcion y planes
- Configuracion del negocio (IGV, datos, meta)
- Notificaciones y busqueda global

Panel SaaS (Super Admin)

- Dashboard con graficos (altas, ingresos, planes)
- Gestion de restaurantes (pago, prueba, suspender)
- Planes y precios
- Perfil del super administrador

3 Instalacion en una computadora nueva

Guia para Windows. Se recomienda **Laragon** porque incluye PHP, MySQL y Apache listos para Laravel (alternativa: XAMPP). Descarga e instala primero estos programas:

Programa	Para que sirve	Donde descargar
Laragon (o XAMPP)	Servidor local: PHP 8.2+, MySQL y Apache	laragon.org / apachefriends.org
Composer	Gestor de paquetes de PHP	getcomposer.org
Node.js LTS	Compilar el frontend (Vite/Tailwind)	nodejs.org
Git (opcional)	Clonar y versionar el proyecto	git-scm.com
HeidiSQL / phpMyAdmin	Administrar la base de datos	incluido en Laragon/XAMPP

Pasos de configuracion (abre la terminal en la carpeta del proyecto):

1. Copia el proyecto a la carpeta del servidor (Laragon: C:\laragon\www\saas_restaurante).
2. Crea la base de datos vacia llamada **saas_restaurante** en phpMyAdmin/HeidiSQL.
3. Crea el archivo de entorno y la clave de la app.
4. Instala las dependencias de PHP y de Node.
5. Configura la conexion a la base de datos en el archivo .env.
6. Ejecuta las migraciones y los datos de demostracion.
7. Compila el frontend y levanta el servidor.

```
# 3) entorno y clave
copy .env.example .env
php artisan key:generate

# 4) dependencias
composer install
npm install

# 5) editar .env -> DB_DATABASE=saas_restaurante DB_USERNAME=root DB_PASSWORD=

# 6) base de datos + datos demo
php artisan migrate --seed
php artisan db:seed --class=DemoDataSeeder

# 7) frontend + servidor
npm run build
php artisan serve -> http://127.0.0.1:8000
```

Credenciales de demostracion: **admin@restaurante.test** / password (restaurante) | **super@saas.test** / password (Super Admin).

4 Hosting web recomendado

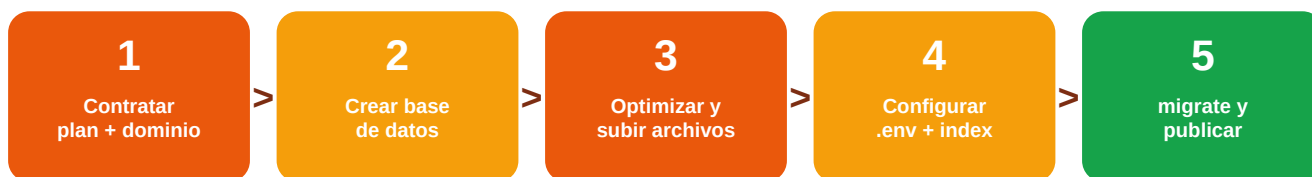
Por ser una app Laravel, necesitas hosting con **PHP 8.2+**, **MySQL**, acceso por consola (SSH) y, idealmente, Composer. Comparativa de opciones populares (precios referenciales, varían según promociones):

Opcion	Tipo	Ideal para	Costo aprox.
Hostinger (Premium/Business)	Hosting compartido	Empezar rapido y economico	Desde ~US\$3/mes
Hostinger VPS (KVM 2)	Servidor VPS	Mas control y rendimiento	Desde ~US\$7-9/mes
DigitalOcean + Laravel Forge	VPS gestionado	Despliegue profesional	Droplet desde ~US\$6 + Forge
Railway / Render	Plataforma (PaaS)	Despliegue automatico por Git	Plan free / desde ~US\$5/mes

RECOMENDADO

Hostinger es la opción más recomendada para este proyecto: económico, popular en Latinoamérica, con panel sencillo (hPanel), soporte para Laravel y opción de crecer a VPS. Para máximo control, usa un **VPS de Hostinger**; para empezar gastando poco, su **hosting compartido** Premium/Business es suficiente.

5 Como subir la app a Hostinger (paso a paso)



Flujo general de publicacion en Hostinger

Despliegue en hosting compartido (hPanel)

1. Contrata un plan (Premium o Business) y asocia tu dominio. Entra al **hPanel**.
2. En hPanel > Bases de datos MySQL, crea la base de datos, el usuario y la contraseña. Anota esos datos.
3. En tu PC, prepara la app para produccion (ver comandos abajo): instala dependencias optimizadas y compila el frontend.
4. Sube el proyecto por **Administrador de archivos** o FTP a una carpeta sobre public_html (ej. /laravel_app).
5. Mueve el contenido de public/ dentro de public_html/ y edita index.php para apuntar a ../laravel_app/vendor/autoload.php y ../laravel_app/bootstrap/app.php.
6. Crea/edita el .env con los datos de la base de datos de Hostinger y APP_URL = tu dominio, APP_ENV=production, APP_DEBUG=false.
7. Abre la consola **SSH** del hPanel y ejecuta la clave, migraciones y enlaces (comandos abajo).
8. Da permisos de escritura a storage/ y bootstrap/cache/ (755) y verifica el .htaccess.
9. Abre tu dominio: la aplicacion deberia cargar. Crea tu cuenta o usa las credenciales sembradas.

```
# En tu PC (preparar produccion)
composer install --optimize-autoloader --no-dev
npm run build
```

```
# En el servidor (SSH de Hostinger)
php artisan key:generate
php artisan migrate --force
php artisan storage:link
php artisan config:cache && php artisan route:cache
```

Alternativa VPS (mas control): conecta por SSH, instala Nginx + PHP-FPM + MySQL, clona el proyecto, ejecuta composer y npm, crea el virtual host apuntando a /public, activa SSL con Certbot y configura las tareas programadas (scheduler) y colas. Para automatizarlo, **Laravel Forge** sobre DigitalOcean/Hostinger VPS hace todo esto por ti.

Recomendacion final: empieza en hosting compartido de Hostinger para validar el negocio; cuando crezca el trafico, migra a un VPS (Hostinger KVM o DigitalOcean + Forge) para mejor rendimiento, colas, cache y despliegues automaticos.